Regulated Search as Complementary Search

From Decision Covers to Self-Bootstrapping Minimax

Abstract

This note develops regulated search from the viewpoint of complementary decision. A decision process is not adequately described as a search followed by a final choice. Rather, it is a synchronized pair of searches over the same unresolved space.

A constructive search generates candidate witnesses, paths, plans, proofs, assignments, or moves. A complementary search generates exclusions, bounds, conflicts, dominance relations, contradictions, equivalence classes, or other reasons why portions of the constructive space need no longer remain active.

The two searches jointly maintain a decision cover

$$W_x = A_t \cup U_t \cup C_t,$$

where A_t is accepted, C_t is closed, and U_t remains unresolved. The key dynamic quantity is the branching surplus

$$\beta_t = e_t - g_t,$$

the imbalance between constructive expansion and complementary closure.

This viewpoint unifies heuristic search, branch-and-bound, graph search, proof search, and minimax. In game playing, alpha–beta pruning becomes a particularly clear instance of complementary search: constructive exploration improves bounds, improved bounds close alternatives, and closure concentrates future search. This feedback naturally suggests a self-bootstrapping form of minimax in which search progressively improves the regulator that controls subsequent search.

1 The Question Behind Search

Suppose a search algorithm is exploring a state space. At some point it decides not to expand a branch.

Why is that legitimate?

In an elementary presentation we might say that the branch is "unpromising." But a correct algorithm needs something stronger. A branch may disappear only because one of two things has happened:

1. it has been resolved positively, or
2. there is a justified reason why it can no longer affect the answer.

This leads to the central idea:

> A good search algorithm does not merely know where to search. It continually constructs reasons for where it no longer has to search.

The second activity is not auxiliary. It is a search in its own right.

2 Constructive and Complementary Search

Consider a decision problem with instance x, witness space W_x, and verification relation

$$V(x, w) \in \{0, 1\}.$$

Define

$$\mathrm{Sol}(x) = \{w \in W_x : V(x, w) = 1\}.$$

A constructive search seeks evidence for

$$\exists w \in W_x \quad \text{such that} \quad V(x, w) = 1.$$

Its objects may be partial witnesses, paths, assignments, proofs, parses, plans, or moves.

But if no witness has yet been found, the computation has a second problem: what can be said about the remaining witness space?

Can some region be excluded? Can a contradiction be derived? Can a bound show that a region cannot improve the current solution? Can a symbolic region be identified with one already solved? Can an entire family of branches be closed by one reason?

This is complementary search.

We therefore introduce two frontiers:

$$\mathcal{F}_t^+ \qquad \text{and} \qquad \mathcal{F}_t^- .$$

The constructive frontier $\mathcal{F}_t^+$ carries candidate positive structure. The complementary frontier $\mathcal{F}_t^-$ carries reasons for closure.

The two frontiers need not contain the same kind of objects.

Problem	Constructive object	Complementary object
SAT	partial assignment	clause conflict / nogood
Shortest path	partial path	lower bound / dominance
Branch-and-bound	candidate region	upper-bound certificate
Proof search	proof branch	contradiction / failed subgoal
Dynamic programming	subproblem	equivalence / dominance
Minimax	candidate continuation	value bound / cutoff

This asymmetry is essential. Complementary search is not necessarily search for a "negative witness" of the same form as the positive one.

3 Decision as a Cover

At time t, partition the witness space into three regions:

$$W_x = A_t \cup U_t \cup C_t.$$

Here A_t is the accepted region, U_t the unresolved region, and C_t the closed region.

For an exact decision process, soundness requires

$$A_t \subseteq \mathrm{Sol}(x), \qquad C_t \cap \mathrm{Sol}(x) = \varnothing.$$

The unresolved region is

$$U_t = W_x \setminus (A_t \cup C_t).$$

This cover is the true state of the decision process.

Constructive search acts by enlarging A_t, or by refining the description of U_t. Complementary search acts by enlarging C_t.

The decision conditions are then

$$A_t \neq \varnothing \quad \Longrightarrow \quad \text{accept},$$

and

$$A_t = \varnothing, \qquad U_t = \varnothing \quad \Longrightarrow \quad \text{reject}.$$

The computation is therefore the progressive resolution of U_t.

4 Complementary Closure as a Running Invariant

Complementary search should not be understood merely as what happens at the end of an unsuccessful constructive search.

At every stage, every live symbolic region must have an explicit status:

$$\text{accepted}, \quad \text{closed}, \quad \text{split}, \quad \text{bounded}, \quad \text{or still unresolved}.$$

Thus the constructive and complementary frontiers are synchronized when they maintain a sound cover

$$W_x = A_t \cup U_t \cup C_t$$

after every update.

A branch is not simply forgotten. It is either retained as unresolved or closed for a reason.

A branch removed without a complementary reason is not regulation. It is merely discard.

5 Symbolic Frontiers

The frontiers need not enumerate individual witnesses.

Let a constructive symbolic node a denote a region

$$\mathcal{L}(a) \subseteq W_x.$$

For

$$\mathcal{F}_t^+ = \{a_1, \ldots, a_k\},$$

define

$$\mathcal{L}(\mathcal{F}_t^+) = \bigcup_{a \in \mathcal{F}_t^+} \mathcal{L}(a).$$

Similarly, let a complementary object c denote a closed region

$$\mathcal{L}^-(c) \subseteq W_x.$$

For

$$\mathcal{F}_t^- = \{c_1, \ldots, c_m\},$$

the closed region is

$$C_t = \bigcup_{c \in \mathcal{F}_t^-} \mathcal{L}^-(c).$$

The unresolved region represented by the two frontiers is therefore

$$U_t = \mathcal{L}(\mathcal{F}_t^+) \setminus C_t.$$

This equation is fundamental.

The constructive frontier tells us what may still contain a solution.

The complementary frontier tells us which part of that live region has already been shown irrelevant, impossible, dominated, or otherwise closed.

6 Expansion and Closure

Let

$$e_t$$

be the constructive expansion gain: the amount of new live search space created or exposed by constructive search.

Let

$$g_t$$

be the complementary closure gain: the amount of search space eliminated by sound closure.

Define the branching surplus

$$\boxed{\beta_t = e_t - g_t.}$$

This is more informative than the raw branching factor.

If

$$\beta_t > 0$$

persistently, unresolved structure accumulates.

If

$$\beta_t < 0$$

persistently, the process may be collapsing alternatives faster than it can test them.

The interesting regime is near balance:

$$e_t \approx g_t.$$

The system continuously generates new possibilities and continuously constructs reasons to eliminate them.

This is the dynamic core of regulated search.

7 Three Regimes

The coupled search admits three qualitative regimes.

7.1 Greedy collapse

If closure dominates too aggressively,

$$g_t \gg e_t,$$

then

$$|U_t|$$

shrinks rapidly.

The computation is cheap, but potentially brittle. It commits before enough evidence has accumulated to justify the collapse.

7.2 Exhaustive dispersion

If expansion dominates,

$$e_t \gg g_t,$$

then

$$|U_t|$$

grows.

The machine preserves possibilities but loses computational control.

7.3 Regulated metastability

The useful regime is intermediate:

$$e_t \approx g_t.$$

Alternatives are created and destroyed at comparable rates.

The unresolved region remains large enough to preserve meaningful competing possibilities, but small enough for the observer to maintain.

8 The Square-Root Band

Let

$$w(n)$$

denote an effective width of unresolved symbolic structure for a problem of scale n.

Suppose regulation requires pairwise interactions among active unresolved objects. Then a natural relational cost is

$$O(w(n)^2).$$

The scale

$$w(n) = O(\sqrt{n})$$

has the property

$$w(n)^2 = O(n).$$

Hence the square-root regime is structurally distinguished: it supports a nontrivial amount of unresolved symbolic structure while keeping pairwise interaction linear in the problem scale.

The important point is that the square-root band is not merely a bound on the constructive frontier.

It is better understood as a band for the coupled unresolved process

$$\mathcal{F}_t^+ \longleftrightarrow \mathcal{F}_t^-,$$

or equivalently for the residual region

$$U_t.$$

The real question is:

How much unresolved structure can remain while expansion and closure stay synchronized?

9 Adaptive Search Reinterpreted

Classical heuristic search is usually presented as selection:

$$x_t = \arg\min_{x \in \mathcal{F}_t} f(x),$$

or, for A*,

$$f(x) = g(x) + h(x).$$

But a regulated interpretation is broader.

The heuristic has two possible roles.

First, it can guide constructive expansion:

where should search continue?

Second, it can support complementary closure:

where is further search no longer necessary?

Thus a useful search procedure couples two questions:

Where should I search?	and	Where can I justify not searching?

10 Branch-and-Bound as the Canonical Example

Consider maximization.

Constructive search tries to improve the incumbent value

$$L.$$

For a region R, complementary search computes an upper bound

$$U(R).$$

If

$$U(R) \leq L,$$

then R is closed.

Now observe the feedback:

better constructive solution $\Longrightarrow L \uparrow \Longrightarrow$ more closure $\Longrightarrow$ fewer unresolved regions $\Longrightarrow$ more focused construct

Therefore:

$$\boxed{\text{construct} \rightarrow \text{improve bound} \rightarrow \text{close} \rightarrow \text{focus} \rightarrow \text{construct better.}}$$

This is already a self-reinforcing search system.

11 Minimax as Adversarial Complementary Search

For a game state s,

$$V(s) = \begin{cases} \max\limits_{a} V(T(s,a)), & \text{MAX}, \\ \min\limits_{a} V(T(s,a)), & \text{MIN}. \end{cases}$$

Naive minimax expands the adversarial completion tree.

Practical minimax survives only because large portions of that tree are complementarily closed.

At a MAX node, constructive search seeks evidence that some move achieves a high value.

At the same time, complementary search seeks evidence that other moves cannot change the decision.

Thus, schematically,

$$\mathcal{F}_t^+ = \{\text{candidate continuations capable of supporting a move}\},$$

while

$$\mathcal{F}_t^- = \{\text{bounds and reasons closing competing continuations}\}.$$

The decision is not produced by the constructive frontier alone.

It emerges from synchronization between constructive support and complementary closure.

12 Alpha–Beta as Complementary Search

Alpha–beta pruning makes the duality explicit.

At a MAX node, α is a lower bound on the value already achievable.

At a MIN node, β is an upper bound on what the opponent can force.

When

$$\alpha \geq \beta,$$

a remaining branch can no longer affect the minimax decision and is closed.

The point is not simply that alpha–beta prunes.

It constructs a reason for closure.

This gives the feedback loop

constructive search → better value bound → complementary cutoff → smaller unresolved frontier → deeper co

So alpha–beta is a concrete instance of synchronized complementary search.

13 Move Ordering and Search Acceleration

Move ordering now acquires a deeper interpretation.

A good move searched early may rapidly increase α.

That does not merely improve the constructive side.

It strengthens the complementary side by allowing more cutoffs.

Therefore move ordering accelerates closure.

Conversely, stronger closure reduces the number of live branches and permits more depth on the remaining candidates.

Hence:

better ordering $\Longrightarrow$ stronger early bounds $\Longrightarrow$ more closure $\Longrightarrow$ more depth $\Longrightarrow$ better bounds.

The process bootstraps itself.

14 Transpositions as Complementary Compression

Suppose two different search trajectories reach the same residual position.

Without memory, the search treats them as separate branches.

A transposition table recognizes

$$s_i \equiv s_j$$

with respect to the residual decision problem.

One branch can then be replaced by a previously computed summary.

This is complementary closure by equivalence.

The search does not prove the branch impossible.

It proves that it contains no unresolved information not already represented elsewhere.

Hence complementary search includes not only exclusion but also compression.

15 Quiescence and the Residual

A fixed search depth is not in itself a decision criterion.

At the nominal horizon, the residual uncertainty may still be structurally large.

Quiescence search asks whether the unresolved region has a stable form.

Thus the stopping condition should not merely be

$$d = d_{\max},$$

but more generally something like

$$R_t \leq \varepsilon,$$

where R_t measures the remaining decision residual.

This connects stopping directly to complementary closure.

16 From Search to a Self-Bootstrapping Regulator

A fixed search algorithm uses a fixed regulation policy.

A self-bootstrapping search system instead allows the search itself to modify the policy that governs future expansion and closure.

Let

$$\mathcal{R}_t$$

be the regulator at time t.

Then

$$\mathcal{R}_{t+1} = \Phi(\mathcal{R}_t, \mathcal{D}_t),$$

where $\mathcal{D}_t$ contains evidence generated by search.

This evidence may include:

- which moves caused large changes in minimax bounds;
- which branches were repeatedly closed with little exploration;
- which positional features predicted useful cutoffs;
- which branches generated persistent unresolved surplus;
- which transpositions repeatedly compressed the frontier;
- which search depths were insufficient because the residual remained high.

The regulator can therefore learn both

$$\text{where expansion is valuable}$$

and

$$\text{where closure is likely.}$$

This is stronger than learning a static evaluation function.

It is learning how to allocate the decision process itself.

17 Feasible Self-Bootstrapping Minimax

At time t, maintain

$$\mathcal{F}_t^+, \qquad \mathcal{F}_t^-, \qquad U_t, \qquad \mathcal{R}_t.$$

The loop is:

1. expand selected constructive continuations;
2. derive new bounds, conflicts, equivalences, or other closure objects;
3. update A_t, U_t, and C_t;
4. measure expansion e_t, closure g_t, and surplus
$$\beta_t = e_t - g_t;$$
5. regulate search so that unresolved structure remains in a feasible band;
6. use the consequences of this search to update $\mathcal{R}_t$;
7. continue until one move is sufficiently supported and competing alternatives are sufficiently closed.

The machine therefore performs

$$\boxed{\text{search} \to \text{closure} \to \text{measurement} \to \text{regulation} \to \text{better search.}}$$

18 The Main Conceptual Chain

The lecture can be summarized by

$$\boxed{\text{Search} \to \text{Constructive Frontier} \leftrightarrow \text{Complementary Frontier} \to \text{Decision Cover} \to \text{Residual} \to \text{Regulation.}}$$

The branching quantity is not simply raw branching:

$$\boxed{\beta_t = \text{constructive expansion} - \text{complementary closure.}}$$

The feasible regime is not simply a narrow search tree.

It is a regime in which unresolved structure remains bounded because expansion and closure stay synchronized.

The practical lesson is:

> Search becomes powerful when it learns not only how to generate promising possibilities, but how to generate increasingly strong reasons for closing the rest.

For game playing, this leads naturally from heuristic search to alpha–beta, from alpha–beta to regulated minimax, and from regulated minimax to a self-bootstrapping search machine.